UTCH BIS

Analysis Report on the Relationship Between Integration and Continuous Delivery with Software Development Planning

Ing. Ramses Requena

Student: Andres Muñoz

ID:6522150058

Date: Feb 11, 2025

INDEX

1. Introduction

2. Foundational Definitions and Interdependencies

    2.1 Continuous Integration (CI): The Quality Gate

    2.2 Continuous Delivery (CD): The Reliability Engine

3. Deep Dive: The Impact of Initial Planning

    3.1 Architecture as the Enabler of Velocity

    3.2 Designing for Automation

4. Conclusion and Reflection

1. Introduction

The boundary between planning and execution has blurred. This report analyzes how Continuous Integration (CI) and Continuous Delivery (CD) are no longer mere post-development tasks but are instead foundational pillars that must be engineered during the initial planning phase. Failure to align these strategies early often results in Automated Chaos where fast pipelines simply deliver poor-quality software at a higher velocity.

2. Foundational Definitions and Interdependencies

2.1 Continuous Integration (CI): The Quality Gate

CI is the technical manifestation of the Fail Fast philosophy. It requires a planning strategy that defines:

- **Trunk-Based Development vs. GitFlow:** Planning the branching strategy determines how the CI server (e.g., Jenkins or GitLab CI) will trigger builds.
- **Automated Verification:** Establishing the Definition of Done (DoD) to include a minimum of 80% code coverage ensures quality from the start.

2.2 Continuous Delivery (CD): The Reliability Engine

CD ensures that the software is always in a release-ready state. This requires:

- **Environment Parity:** Planning for Infrastructure as Code (IaC) to ensure development, testing, and production environments are identical, preventing configuration drift.
- **Deployment Orchestration:** Deciding early whether to use Kubernetes for container orchestration or serverless architectures to manage deployment scales.

3. Deep Dive: The Impact of Initial Planning

3.1 Architecture as the Enabler of Velocity

The software architecture is the skeleton upon which CI/CD sits.

- **Decoupling:** During planning, architects must design for low coupling. If a system is a Distributed Monolith, the pipeline will experience frequent failures due to side effects, regardless of automation tools.
- **API-First Design:** Planning for stable APIs allows different teams to run independent CI/CD pipelines, significantly increasing organizational velocity.

3.2 Designing for Automation

Automation is a design constraint, not an afterthought.

- **Testability:** If code is not planned to be testable (e.g., using Dependency Injection), automated unit testing becomes impossible.
- **Security (DevSecOps):** Planning must include automated security scans (SAST/DAST) within the CI pipeline to ensure security is a continuous process.

4. Conclusion and Reflection

The relationship between integration, delivery, and planning is cyclical. Planning defines the constraints of the CI/CD pipeline, while the data from the pipeline informs the next planning cycle. To achieve a high-performing software team, CI/CD must be viewed as a product feature designed during project inception. By prioritizing architecture and automation early, organizations move from simple "coding" to continuous value delivery.